FYS3150/4150 Project 2

Johannes Schäfers, Ferdinand Nilius, Michał Kiedrzyński
(Dated: October 27, 2026)

All code for Project 2 can be found under: https://github.uio.no/johascha/CompPhys/tree/master/Project_2.

PROBLEM 1

In order to rewrite the equation:

$$\gamma \frac{d^2u}{dx^2} = -Fu(x) \tag{1}$$

into a scaled form, where

$$\hat{x} = \frac{x}{L},$$

the second derivative of $u(x)$ with respect to the new variable $\hat{x}$ must be found. This can be done as follows:

$$\frac{d^2u(x)}{d^2x} = \frac{d^2u(\hat{x})}{d^2\hat{x}} \cdot \frac{d\hat{x}}{dx} \cdot \frac{d\hat{x}}{dx} = \frac{d^2u(\hat{x})}{d^2\hat{x}} \cdot L^{-2}.$$

Substituting this into equation (1) and reforming the equation yields:

$$\frac{d^2u(\hat{x})}{d^2\hat{x}} = -\frac{F \cdot L^2}{\gamma} u(\hat{x}) = -\lambda u(\hat{x}).$$

Here, λ is used as a new constant that incorporates the term $\frac{F \cdot L^2}{\gamma}$.

PROBLEM 2

The code consists of two main parts. Implementing the 6 x 6 matrix and solving the eigenvalues as well as the eigenvectors. Therefore, we decided to use the armadillo library, which calculates the eigenvectors and eigenvalues of A. A is given as the following matrix:

$$A = \frac{1}{h^2} \begin{bmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{bmatrix}. \tag{2}$$

Running the algorithm shows that the analytical solution is identical to the numeric solution (up to 6 decimal places, see Table I), which was calculated using the Armadillo library. Differences between the two solutions only show, after 14 decimal places. These are likely caused by rounding errors.

j	Analytical λ_j	Numerical λ_j	Absolute Difference
1	9.705050×10^0	9.705050×10^0	3.55×10^{-15}
2	3.689799×10^1	3.689799×10^1	2.13×10^{-14}
3	7.619294×10^1	7.619294×10^1	1.42×10^{-14}
4	1.198070×10^2	1.198070×10^2	2.84×10^{-15}
5	1.591020×10^2	1.591020×10^2	0.00×10^0
6	1.862949×10^2	1.862949×10^2	5.68×10^{-14}

TABLE I: Comparison of analytical and numerical eigenvalues for $N = 6$.

PROBLEM 3

Task a)

Since the matrix remains the same throughout the algorithm, it can be used as a constant. The algorithm uses two loops, one for the rows and one for the columns, to create a grid search, using the matrix element $k = 0, l = 1$ as an initial value, while checking all other elements from the bottom upward afterwards. If the algorithm finds a larger absolute value, the variable containing the maximum value of $\mathbf{A}$, as well as the two variables that hold the indices of the said element, are updated. After all elements above the diagonal have been checked, the algorithm returns the largest absolute value.

Task b)

The given matrix $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & -0.7 & 0 \\ 0 & -0.7 & 1 & 0 \\ 0.5 & 0 & 0 & 1 \end{bmatrix}$$

Obviously, the absolute highest value without the main diagonal is -0.7, the algorithm in the previous section delivers successfully the same value.

PROBLEM 4

Task a)

The implementation of the Jacobi Rotation Algorithm can be described as follows: Starting with the variable τ, which is given as:

$$\tau = \frac{A_{ll} - A_{kk}}{2A_{kl}}$$

Where A_{kl} is the largest non diagonal element of the matrix A. Now the goal is to select the smaller root of the quadratic equation for the tangent parameter t in the Jacobi Rotation Algorithm. It uses different formulations for positive and negative tau to prevent numerical loss of significance.

$$t = \tan(\tau) = \begin{cases} \frac{1}{\tau + \sqrt{1+\tau^2}} & \text{if } \tau \geq 0 \\ \frac{-1}{-\tau + \sqrt{1+\tau^2}} & \text{if } \tau < 0 \end{cases}$$

Out of this quadratic equation the solution for the parameter c and s are the following. Hereby $c = \cos(\theta)$ and $s = \sin(\theta)$ are the roation parameters.

$$c = \frac{1}{\sqrt{1+t^2}}$$

$$s = c \cdot t$$

In the following steps, the matrix elements are updated by the algorithm. Especially the two diagonal elements at indices (k, k) and (l, l) are altered by the rotation.

$$A'_{kk} = A_{kk}c^2 - 2A_{kl}cs + A_{ll}s^2$$

$$A'_{ll} = A_{ll}c^2 + 2A_{kl}cs + A_{kk}s^2$$

The largest off-diagonal elements are forced to zero, which is the main objective of the rotation step.

$$A'_{kl} = A'_{lk} = 0.0$$

Updating all other elements in row l and column l to account for how the rotation mixes the indices k and l

$$A'_{ik} = A'_{ki} = A_{ik}c - A_{il}s \quad \text{for } i \neq k, l$$

Same for l

$$A'_{il} = A'_{li} = A_{il}c + A_{ik}s \quad \text{for } i \neq k, l$$

Finally, the eigenvector matrix $\mathbf{R}$ is updated. Each column of $\mathbf{R}$ represents an eigenvector corresponding to an eigenvalue in $\mathbf{A}$. You have to apply the same rotations to $\mathbf{R}$ as you did to $\mathbf{A}$.

$$R'_{ik} = R_{ik}c - R_{il}s \quad \text{for all } i$$
$$R'_{il} = R_{il}c + R_{ik}s \quad \text{for all } i$$

A short summary of the Jacobi Rotation Algorithm as pseudo-code can be described as the following:

Algorithm 1 Jacobi Rotation Algorithm

Require: Symmetric matrix $A \in \mathbb{R}^{n \times n}$, tolerance ε
Ensure: Diagonal matrix D and eigenvector matrix R

$R \leftarrow I_n$
while $\max_{i \neq j} |A_{ij}| > \varepsilon$ **do**
  $(k, l) \leftarrow \arg\max_{1 \leq i < j \leq n} |A_{ij}|$
  $\tau \leftarrow \dfrac{A_{ll} - A_{kk}}{2A_{kl}}$
  if $\tau \geq 0$ **then**
    $t \leftarrow \dfrac{1}{\tau + \sqrt{1 + \tau^2}}$
  else
    $t \leftarrow -\dfrac{1}{-\tau + \sqrt{1 + \tau^2}}$
  $c \leftarrow \dfrac{1}{\sqrt{1 + t^2}}$
  $s \leftarrow tc$
  $a \leftarrow A_{kk}, \quad d \leftarrow A_{ll}, \quad b \leftarrow A_{kl}$
  $A_{kk} \leftarrow a - 2tb + t^2 d$
  $A_{ll} \leftarrow d + 2tb + t^2 a$
  $A_{kl} \leftarrow 0$
  $A_{lk} \leftarrow 0$
  for $i = 1, 2, \ldots, n$, $i \neq k, l$ **do**
    $a_i \leftarrow A_{ki}, \quad d_i \leftarrow A_{li}$
    $A_{ki} \leftarrow a_i - td_i$
    $A_{ik} \leftarrow A_{ki}$
    $A_{li} \leftarrow d_i + ta_i$
    $A_{il} \leftarrow A_{li}$
  for $i = 1, 2, \ldots, n$ **do**
    $r_i \leftarrow R_{ki}, \quad q_i \leftarrow R_{li}$
    $R_{ki} \leftarrow r_i - tq_i$
    $R_{li} \leftarrow q_i + tr_i$
$D \leftarrow A$
return D, R

Task b)

Using the same matrix as in Equation 2, the written algorithm generates the expected eigenvalues and eigenvectors. The outputs are saved to a .txt file, where they can be compared. The calculated eigenvalues can be seen in the table below:

j	Analytical λ_j	Numerical λ_j	Absolute Difference
1	9.705050×10^0	9.705050×10^0	2.66×10^{-14}
2	3.689799×10^1	3.689799×10^1	6.39×10^{-14}
3	7.619294×10^1	7.619294×10^1	1.42×10^{-14}
4	1.198070×10^2	1.198070×10^2	2.84×10^{-14}
5	1.591020×10^2	1.591020×10^2	1.42×10^{-13}
6	1.862949×10^2	1.862949×10^2	1.42×10^{-13}

TABLE II: Comparison of analytical and numerical eigenvalues for $N = 6$.

From the absolute differences between the newly calculated numerical eigenvalues and the analytical eigenvalues listed in table II, it can be seen that the new algorithm performs slightly worse than the implementation in the Armadillo library. Similarly, the eigenvectors, which are also output into a .txt file, show only minor differences, with the largest individual per element difference being of magnitude 10^{-16}, when the eigenvectors are normed to length 1 in their L2-norm. Important to note is that the eigenvectors can be flipped, since the code does not check the orientation of the eigenvectors.

PROBLEM 5

a)

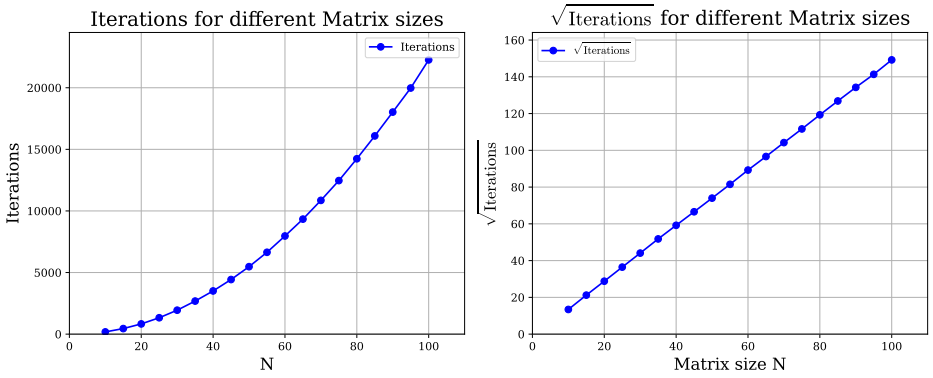


FIG. 1: Comparison between the matrix size and the number of iterations needed to find the eigenvectors for the tridiagonal matrix (left). Comparison between the matrix size and the square root of the number of iterations needed to find the eigenvectors for the tridiagonal matrix (right).

In the plot above (Figure 1, left), it can be seen how the number of similarity transformations needed for the Jacobi Rotation Algorithm develops based on the size of the matrix. For this exercise the matrix, which has been presented in equation 2, was used and iterations were done until the largest off-diagonal element was smaller than 10^{-20}. Taking the square root of the number of iterations and creating a similar plot (Figure 1, right) reveals a linear relationship. Therefore the assumption can be made that the number of iterations rises quadratically with the dimensions of the quadratic matrix. This is contradicted by [1], who argue that it takes $log(n)$ sweeps, where a sweep consists of $\frac{n(n-1)}{2}$ rotations. However, this scaling behavior, which would in total be equal to $\mathcal{O}(log(n)n^2)$ is not properly seen here as the plots only display the number of iterations for matrix sizes of up to 100×100, where the logarithmic factor barely has an influence on the overall result.

b)

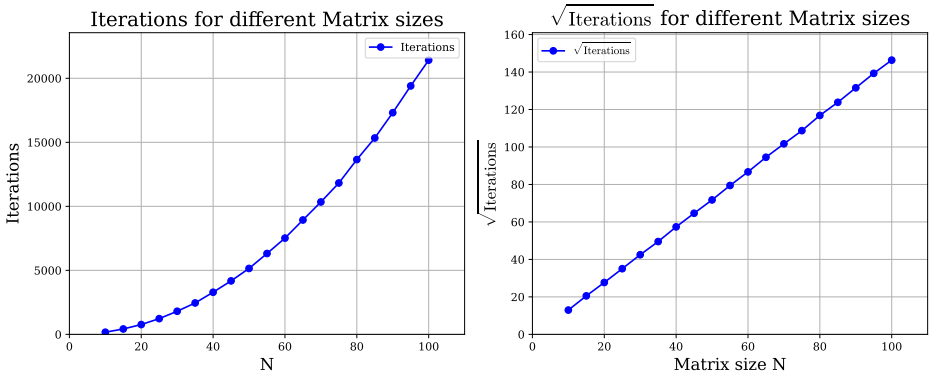


FIG. 2: Comparison between the matrix size and the number of iterations needed to find the eigenvectors for the random dense matrix (left). Comparison between the matrix size and the square root of the number of iterations needed to find the eigenvectors for the random dense matrix (right).

When repeating the same process with a dense, random, and symmetric matrix, the scaling of the number of iterations with increasing matrix size remains the same. This phenomenon can be explained by the destruction of the tridiagonal structure under the rotations applied during the algorithm. As the algorithm does its first iteration on a tridiagonal, it will not only update the elements on the sub- and super-diagonal, but of elements on the same column or row as the largest element. Therefore after even a single step the tridiagonal structure is lost and the scaling is similar to a dense matrix.

PROBLEM 6

Using the following discretization:

$$\frac{-v_{i+1} + 2v_i - v_{v-1}}{h^2} = \lambda v_i$$

we define the matrix $\mathbf{A_N} : (n-1 \times n-1)$ as:

$$\mathbf{A_N} = \frac{1}{h^2} \begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & -1 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{bmatrix}.$$

Then, we apply the Jacobi solver from the previous tasks to compute the matrix's eigenvectors and eigenvalues. Figures 3 and 4 present the comparison of the calculated numerical result to the analytical solution. They present the eigenvectors associated with the three lowest eigenvalues, for $n = 10$ and $n = 100$ respectively. The resulting functions resemble the first three harmonics.

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. (Johns Hopkins University Press, Baltimore, MD, USA, 2013).

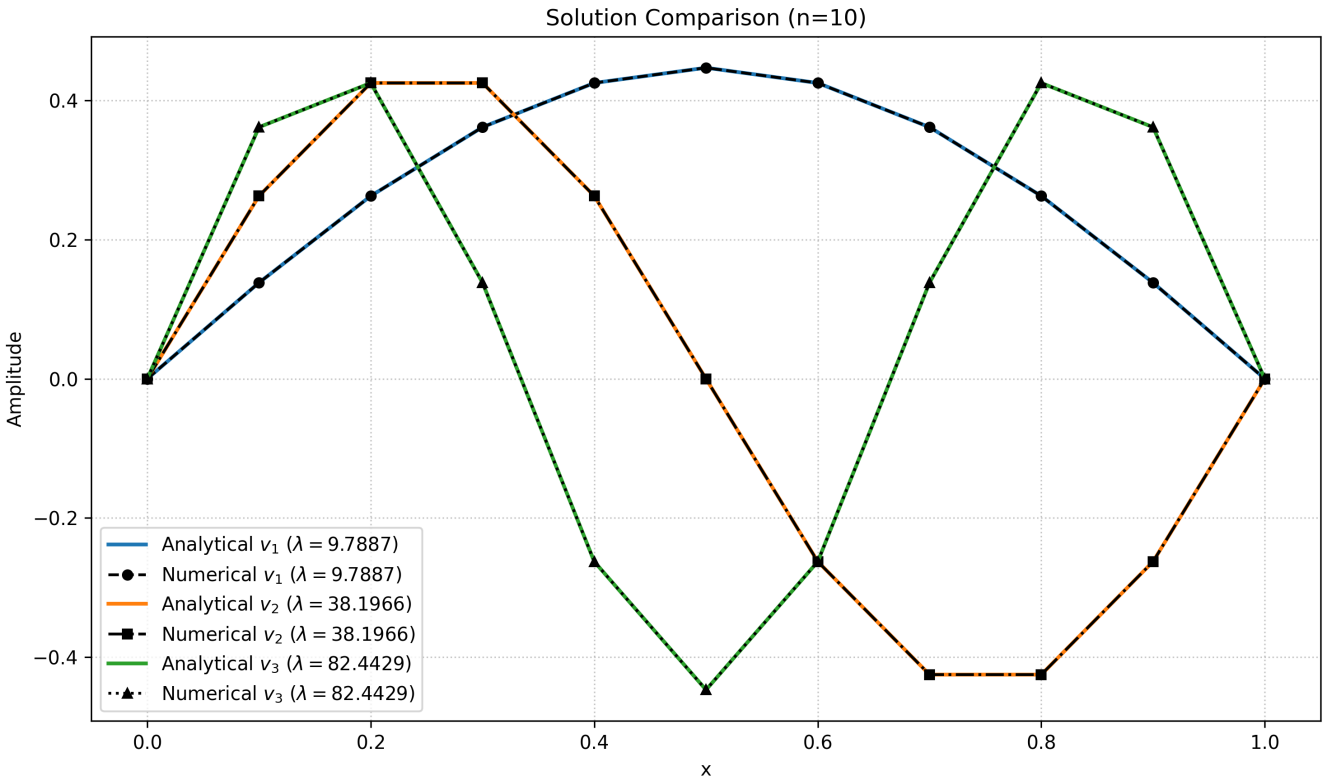


FIG. 3: Comparison of analytical and numerical solutions for eigenvectors with three lowest eigenvalues (n=10)

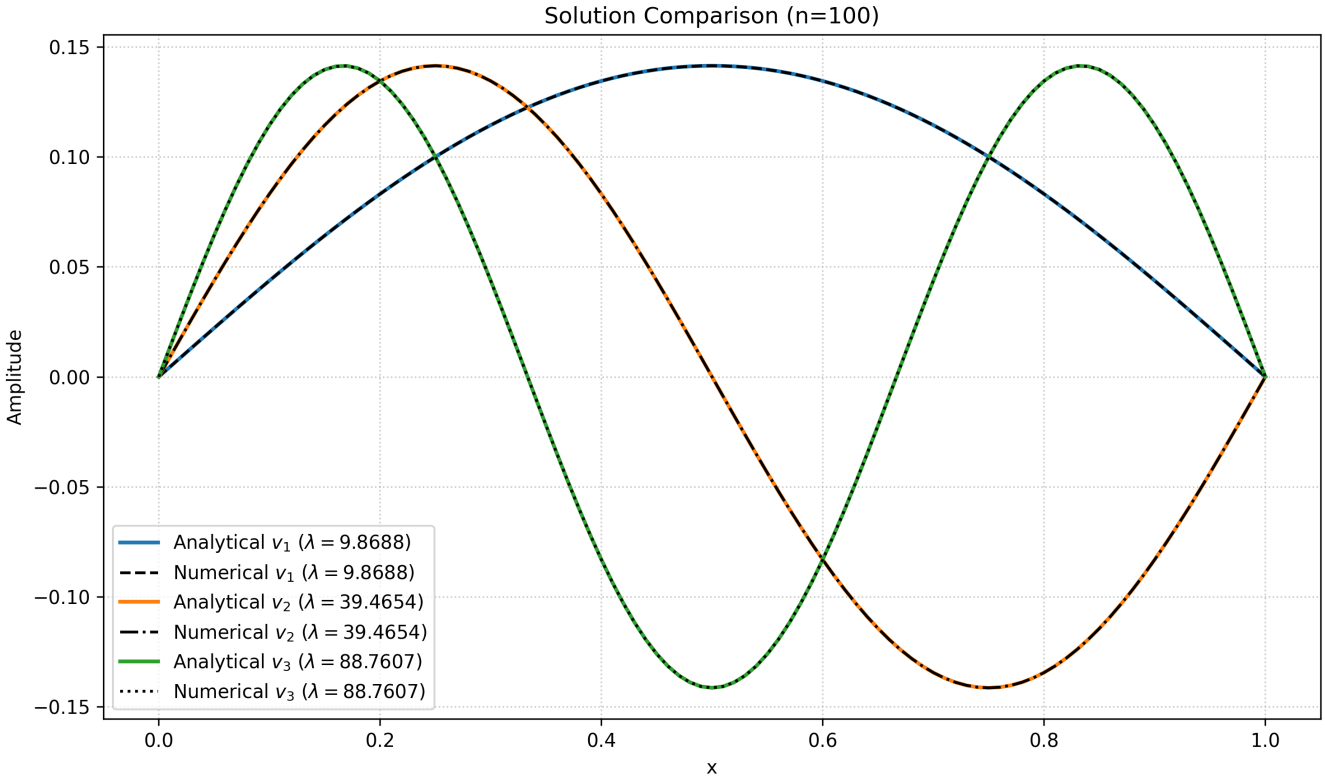


FIG. 4: Comparison of analytical and numerical solutions for eigenvectors with three lowest eigenvalues (n=100)